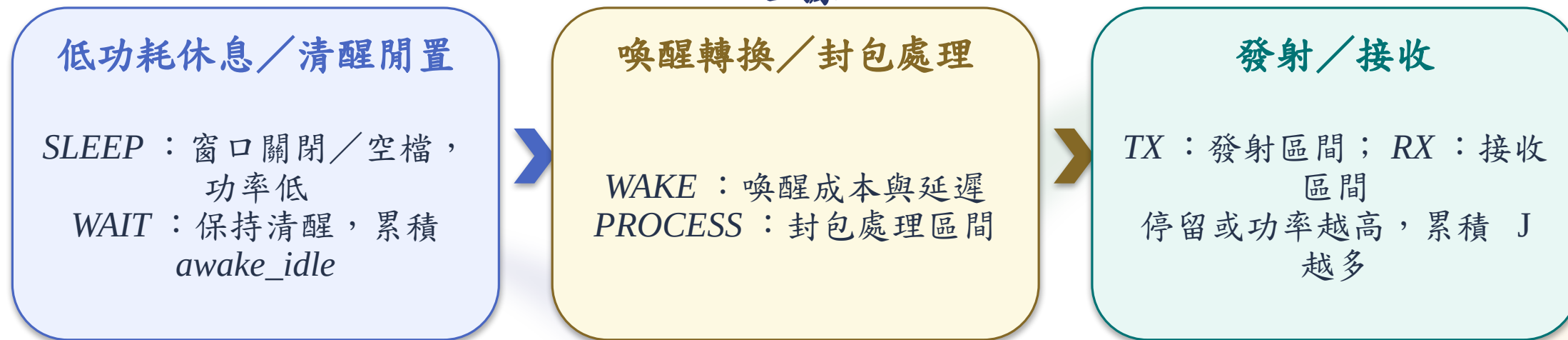


無線狀態與停留時間（Radio state）

策略動作 *action* 只記一次；狀態停留區間 *state interval* 才留下時間與能量證據



完整讀法：動作 *action* → 狀態區間 *state interval* → 能量分桶 *energy bucket*

同一個動作只標記一次；停留秒數 *duration* 才形成 J

無線狀態 *radio_state*（欄位） | 來源：runner events | 作用：標記停留區間
單位：state + s | 判讀：對回 energy bucket

空檔策略的服務與能量

固定工作與服務窗口，只改一個空檔策略值，檢驗它如何改變狀態、服務與 J

為何做

同一份工作與窗口
檢驗空檔策略是否改變服務
與 J

機制：休息／閒置

SLEEP：低功耗，可能付
WAKE
WAIT：清醒閒置，可能少喚
醒

判讀順序

先讀交付、期限、新鮮度
再讀端點 J；低 J 不等於
成功

固定不變：*student_policy.py* 的
PACE_GAP_STEPS=2、*scenario*／*seed*／
traffic／*window*、Lab B/C

唯一變更：*lab-a-pace-rest*／*REST_DURING_GAP*
SLEEP → *WAIT*

成功：*status=OK* + *result_path*
讀 *result.json* + *endpoint-replay.json*
順序：服務→封包→狀態→J

服務布林狀態 *service_pass* (欄位) | 來源：*result summary* | 作用：*required packet* + 期限 + *freshness*
單位：*true*／*false* | 判讀：先判 *service*，再讀 J

封包生命週期：嘗試不等於交付

封包結果 *packet outcome* 要走完整生命週期；送出次數只是其中一段



嘗試封包數 *attempted_packets* (欄位) | 來源：result summary | 作用：統計進入無線嘗試
單位：count | 判讀：對照 *retry*、*expired*、*delivered*

服務門檻先於端點能量

先固定同一比較邊界，再判定服務門檻 *service gate*，最後讀端點能量 *J*

1 | 固定比較邊界

情境 *scenario*、種子 *seed*、流量 *traffic*、窗口 *window*
端點範圍 *endpoint scope* 必須一致

2 | 服務門檻

交付位元 *delivered_bits*、
期限 *deadline_pass*
新鮮度 *freshness_status* 共同形成 *gate*

3 | 端點能量 *J*

欄位 *endpoint_energy_j* 來自整段 *state ledger*
單位 *J*；只在同一邊界內比較

服務通過 *service_pass=true*：才可在相同工作邊界內比較 *J*

服務未通過 *service_pass=false*：*J* 只描述能量／服務取捨

新鮮度狀態 *freshness_status*（欄位） | 來源：*result summary* | 作用：*required packet* 是否新鮮
單位：*fresh*／*stale*／*expired*／*N/A* | 判讀：與 *deadline*、*delivery* 決定 *gate*

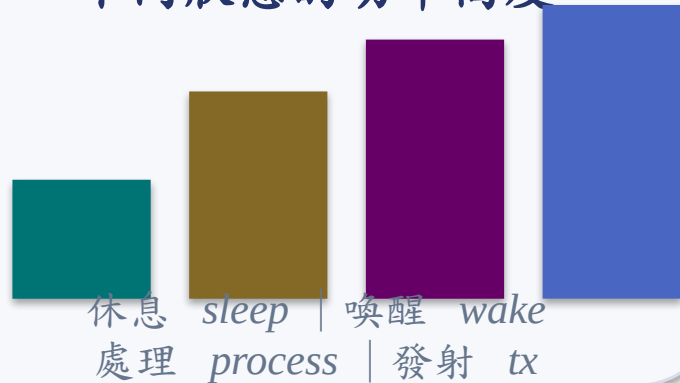
功率 W $\times$ 時間 s 形成能量 J

同一個功率值，停留時間不同就會留下不同的端點能量

功率 P

某個狀態當下的高度
單位：W
例如 *awake_idle* 或 *tx*

不同狀態的功率高度



時間 t

狀態區間 *state interval*
的長度
單位：s
 $P \times t = J$

能量分桶 *bucket* 分開累積：
awake_idle / *sleep* / *wake* / *process* / *tx* / *rx*

固定 10 s 時鐘步長 *clock step*；收發流程另
加入轉換能量

能量分桶 *energy_breakdown_j* (欄位) | 來源：result artifact | 作用：按 state 保存 J
單位：J / bucket | 判讀：拆回 *state interval*

端點能量的累積公式

公式說明端點狀態如何累積；結果產物 *artifact* 提供這次執行的數值

狀態功率 P_s

某個狀態的功率高度
單位：W
來源：energy profile

端點能量 $E_{\text{endpoint}} = \Sigma P_s \times t_s$

分段累積 | 單位：J

停留時間 t_s

狀態區間 *state interval*
單位：s
來源：
STATE_INTERVAL 事件

端點範圍 *endpoint scope*：宣告的 radio / processing 狀態與轉換

不包含衛星 *satellite*、閘道 *gateway*、整體插座 *wall-plug*

端點能量 *endpoint_energy_j* (欄位) | 來源：result summary | 作用：同一 *endpoint boundary* 的總和
單位：J | 判讀：對照 *energy bucket*

端點能量效率的 bit/J 邊界

bit/J 是同一端點範圍內的效率比值；服務門檻仍獨立判定

交付資料 $D_{delivered}$

實際交付的資料量
單位：bit
來源：結果摘要

端點效率 $\eta_E =$
 $D_{delivered} / E_{endpoint}$

bit ÷ J = bit/J

端點能量 $E_{endpoint}$

端點狀態的累積能量
單位：J
來源：結果摘要

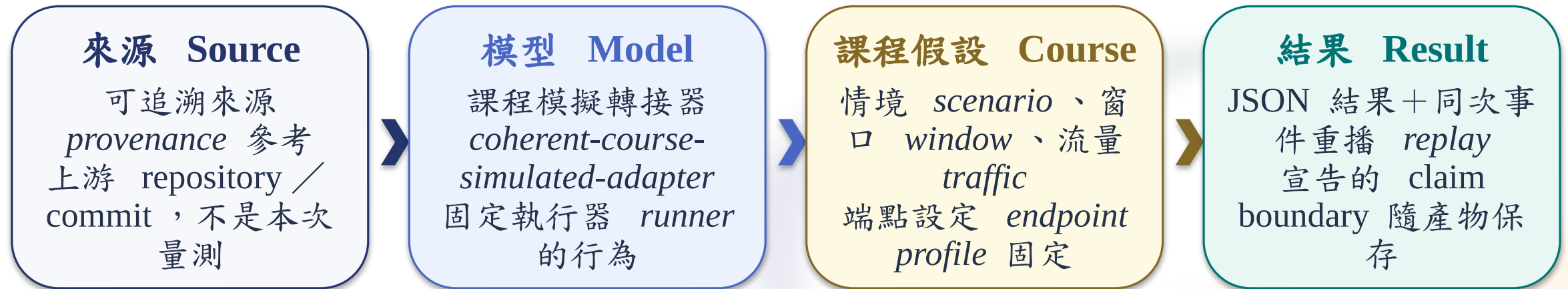
bit/J 高但 $service_pass=false$ ：
只作效率觀察，不作服務成功判定

$service_pass$ 、 $deadline_pass$ 仍獨立判定：
依必要封包與期限讀結果

端點能量效率 $endpoint_energy_efficiency_bits_per_j$ (欄位) | 來源：result summary | 作用：delivered bit ÷ endpoint J
單位：bit/J | 判讀：不取代 service gate

來源、模型、課程假設與結果

同一筆數字要沿來源、模型、固定課程假設與結果產物 *artifact* 回溯



上游執行旗標 *upstream_execution* =
false：這不是上游即時執行

結果型態：一致的模擬結果；所有數值以
本次產物 *artifact* 讀值

執行來源 *runner_provenance* (欄位) | 來源：result artifact | 作用：保存 model / assumption / execution
單位：record | 判讀：設定 claim ceiling

SIMULATED TEACHING DATA | NOT LIVE | NOT MEASURED | NOT CANONICAL-PARITY-VERIFIED

教育部智慧節能網路跨層系統整合教學聯盟

執行、事件重播與工作簿的身分鏈

同一組身分欄位連結結果、重播與工作簿；不是只靠一個情境識別碼



SIMULATED TEACHING DATA | NOT LIVE | NOT MEASURED | NOT CANONICAL-PARITY-VERIFIED

教育部智慧節能網路跨層系統整合教學聯盟

基準 Baseline：固定條件的對照執行

基準 *Baseline* 固定情境、種子與原始策略，才提供可比較的對照



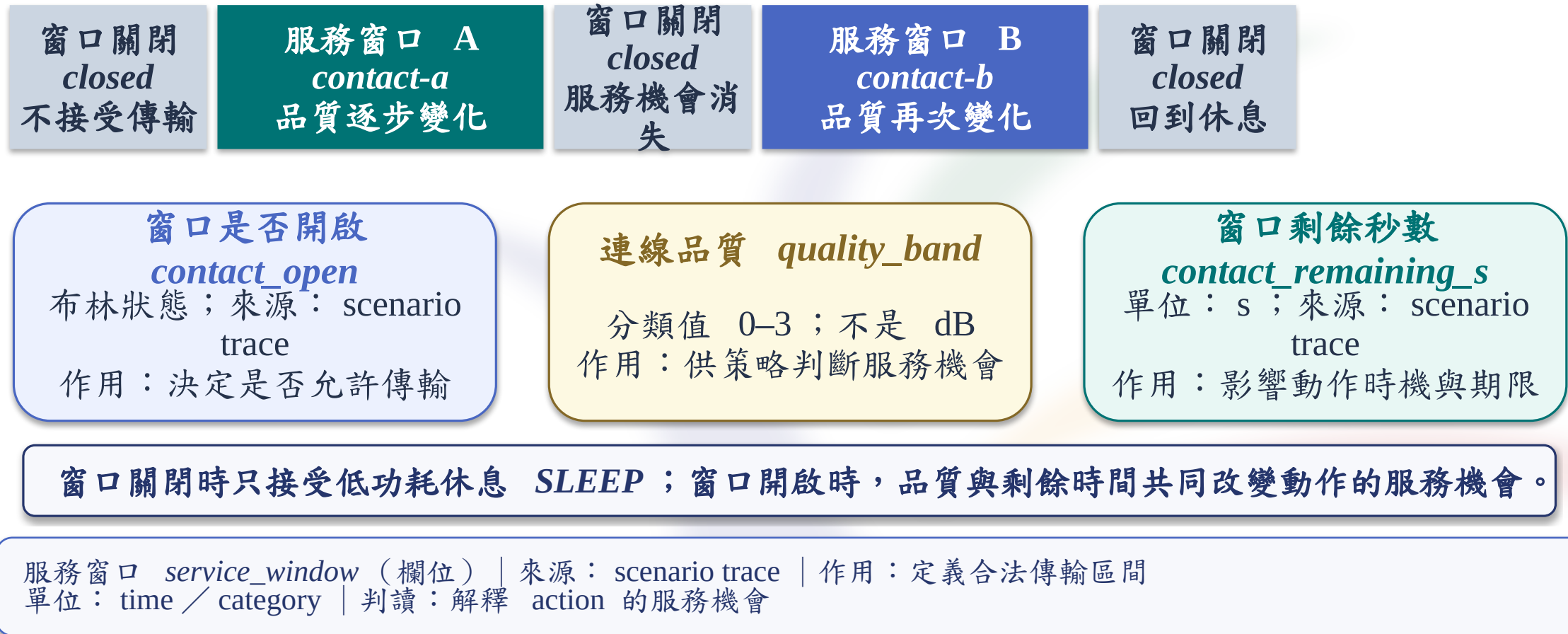
公平條件：兩次結果的身分與端點範圍一致

解釋順序：先回到標記區塊，再讀狀態／封包／服務／J

策略識別 *policy identity* (欄位) | 來源：result policy | 作用：指出 *policy surface*
單位：識別碼 | 判讀：與 *scenario*、*run_id* 比對

LEO 的變動服務窗口軌跡

服務窗口輸入改變合法傳輸機會；策略再依觀察值選擇動作



為何改 REST_DURING_GAP

先說明為何做，再定位檔案、標記區塊、命令與結果欄位

為何做 Why

SLEEP ↔ *WAIT* 先改變
WAKE / *awake-idle* /
J；要驗證 *state* 與
service

檔案與區塊

檔案：
student_policy.py
標記區塊：*lab-a-pace-*
rest

改前 Before

REST_DURING_GAP =
SLEEP
低功耗休息；可能出現
WAKE

改後 After

REST_DURING_GAP =
WAIT
清醒閒置；可能少
WAKE、但多 *awake-*
idle J

固定：*PACE_GAP_STEPS*=2、*scenario* / *seed*
/ *traffic* / *window*、*runner* / *schema*、*Lab*
B/C

成功：*status*=OK + 新 *result_path*；讀同目錄
result.json / *endpoint-replay.json*

完整解釋句：*SLEEP* → *WAIT* 先改變 *state*，再讀 *packet* / *service*，最後才解釋 *endpoint J*；方向可
被結果反駁。

空檔策略欄位 *REST_DURING_GAP*（欄位） | 來源：*policy marked block* | 作用：控制 *gap state*
單位：*SLEEP* / *WAIT* | 判讀：先連 *state*，再讀 *packet* / *service* / *J*

基準決策與對照證據

基準先證明原始策略在同一服務窗口如何決策，建立改前對照

決策分支

contact_open=false → *SLEEP*
steps_since_send < 2 →
REST_DURING_GAP=SLEEP

基準命令

Linux / macOS 終端機
(WSL 也使用 Linux 指令)
bash course.sh run --lab A --
case baseline

參考摘要

endpoint_energy_j=6.92 J
delivered_bits=4,800 bit ;
service_pass=false

固定： *student_policy.py* / *lab-a-pace-rest* ;
PACE_GAP_STEPS=2 、 *REST_DURING_GAP=*
SLEEP ；其他區塊不變

讀取： *result.json* → *summary* / *events* →
endpoint-replay.json ； *baseline* 是 *control* ，不
回答省電。

結果路徑 *result_path* (欄位) | 來源： P042 stdout | 作用：定位 *result.json* + 同 *run replay*
單位： path | 判讀：先核對 *identity*

程式碼改前／改後

定位後只讀 5 行：淡化不變行，高亮唯一變更 *SLEEP* → *WAIT*

改前 Before | 基準

```
# LORA EDITABLE: lab-a-pace-rest
PACE_GAP_STEPS = 2
REST_DURING_GAP = SLEEP
# 其他 marked block 保持原樣
# runner / scenario / schema / JSON 不變
```

唯一變更
➔

改後 After | 候選

```
# LORA EDITABLE: lab-a-pace-rest
PACE_GAP_STEPS = 2
REST_DURING_GAP = WAIT
# 其他 marked block 保持原樣
# runner / scenario / schema / JSON 不變
```

只改 *student_policy.py* 的 *lab-a-pace-rest*： *REST_DURING_GAP*； *PACE_GAP_STEPS=2*、Lab B/C、runner、scenario、schema、JSON 不變。

原因： *WAIT* 先改 *awake-idle state*； P042 run 才看 *WAKE*、*packet*、*service*、*J*。

空檔策略欄位 *REST_DURING_GAP*（欄位） | 來源：*student_policy.py* / *lab-a-pace-rest* | 作用：指定 gap action
單位：*SLEEP* / *WAIT* | 判讀：先連 state，再用 A/B 結果判讀

基準 → 候選 → 隱藏檢查

一條執行帶完成三步：建立基準、凍結候選、用同一策略做隱藏檢查

平台：Linux / macOS 終端機（WSL 也使用 Linux 指令）；Windows 命令提示字元。

01 | 基準 Baseline

不改 policy；A 區塊仍為
SLEEP

Linux / macOS：
bash course.sh run --lab A --case
baseline

Windows：
course.cmd run --lab A --case
baseline

成功：status=OK + result_path
同目錄 result.json / endpoint-
replay.json

02 | 候選 Candidate

完成 P041：A 區塊改
WAIT，再凍結

Linux / macOS：
bash course.sh run --lab A --case
candidate --freeze

Windows：
course.cmd run --lab A --case
candidate --freeze

成功：result_path +
lab-a-frozen.json / .py

03 | 隱藏 Hidden

保持 frozen policy；不再改值

Linux / macOS：
bash course.sh run --lab A --case
hidden

Windows：
course.cmd run --lab A --case
hidden

成功：status=OK + result_path
不建立第二個 freeze

讀取順序：stdout result_path → 同一 generated 目錄 result.json → 配對 endpoint-replay.json → 服務／封包／狀態／J；freeze 缺失就停止 hidden。

SIMULATED TEACHING DATA | NOT LIVE | NOT MEASURED | NOT CANONICAL-PARITY-VERIFIED

教育部智慧節能網路跨層系統整合教學聯盟

同一邊界的改前／改後

開啟 P042 stdout 指出的結果路徑，先核對身分，再讀改前／改後

基準 Baseline

參考： $endpoint_energy_j = 6.92 J$
 $delivered_bits = 4,800 bit$
 $service_pass = false$

身分門檻 Identity gate

情境識別 $scenario_id$ 、種子 $seed$
 策略身分、單位、來源鏈
 同一邊界才可比較

候選 Candidate

參考： $endpoint_energy_j = 8.86 J$
 $delivered_bits = 4,800 bit$
 $service_pass = false$

讀取： $result_path \rightarrow summary / events \rightarrow$ 配對
 replay

改前： SLEEP 的 $state / packet / service$

讀取： $result_path \rightarrow summary / events \rightarrow$ 配對
 replay

改後： WAIT 的 $state / packet / service / J$

解釋：先 $state$ ，再 $packet / service$ ，最後 J ；不能只用較低 J 宣稱成功。

策略識別 $policy_identity$ （欄位） | 來源： P042 result path | 作用：確認 $baseline / candidate policy$
 單位： $identity$ | 判讀：核對 $scenario$ 、 $seed$ 、 $units$

狀態帳本回到端點能量

用事件與能量分桶，把 *WAIT* 的成本拆回可檢查的狀態區間

帳本讀取順序 Ledger

- 1 | 狀態區間 *STATE_INTERVAL* : 找 *WAIT* / *SLEEP* 停留秒數
- 2 | 喚醒 *WAKE* : 確認轉換是否出現
- 3 | 能量分桶 *energy_breakdown_j* : 對回每個 bucket

來源：候選 *result_path* 的 *events*
保留 *state*、起訖秒數、事件能量與 bucket J

能量分桶 *energy_breakdown_j*

醒著閒置 *awake_idle* 6.00 J | 休息 *sleep* 0.04 J
喚醒 *wake* 0.02 J | 處理 *process* 0.16 J
發射 *tx* 2.40 J | 接收 *rx* 0.24 J

讀取： *result_path* → *STATE_INTERVAL* / *WAKE*
→ *energy_breakdown_j*
哪個 interval 改變，就解釋哪個 J bucket

狀態區間事件 *STATE_INTERVAL* (欄位) | 來源：candidate events | 作用：提供 *state*、*time*、*event energy*
單位：state + s + J | 判讀：沿 interval 判讀 *WAIT* 成本

封包結果回到服務判定

嘗試 *attempted* 不等於交付 *delivered* ；服務門檻再決定 J 的解讀層級



期限未通過 *deadline_pass = false*
服務未通過 *service_pass = false*

讀取： *result_path* → *result.json* → 同目錄
endpoint-replay.json
數值不從投影片手填

完整解釋：改 *REST_DURING_GAP* → state → packet outcome → service gate → endpoint J 。

期限通過狀態 *deadline_pass* (欄位) | 來源：result summary | 作用：required packet 是否準時交付
單位：true / false | 判讀：與 delivered / expired / freshness 成 gate